

Plan de lucru — academy

Două faze: întâi aduci toate entitățile la forma lui Book, apoi aplici pattern-urile acolo unde codul le cere.

Proiectul ăsta devine terenul pe care aplici pattern-urile din modulul `design-patterns-csharp`.

Regula: un task per rundă. Build + rulare înainte de fiecare commit. Nu treci la următorul până nu e verde cel curent.

Ce NU se face: nu rescrii tot proiectul. Fiecare task spune explicit ce atingi și ce lași în pace.

Harta lucrului — două faze

Planul are două jumătăți, și ordinea dintre ele nu e negociabilă.

	Ce faci	De ce în ordinea asta
Faza 1	Aduci <code>Course</code> , <code>Enrolment</code> și <code>User</code> la forma lui <code>Book</code>	Copiezi o formă care există deja în cod. Zero decizii de design
Faza 2	Aplici pattern-urile învățate, acolo unde codul le cere	Fiecare pattern intră peste o structură curată; peste cea de acum ar doar muta bug-ul

`Book` e dus până la capăt, de mine, ca model: DTO-uri `record` + `Books/Mappers/BookMapper.cs` , `Books/Repositories/BookRepository.cs` injectat în serviciu, `Common/IEntity` + `Common/ITextMapper<T>` + `Common/Repository<T>` cu mapper injectat. **Deschide fișierele alea înainte să scrii ceva** — sunt răspunsul, nu o sugestie.

Faza 1 — adaptezi tot la modelul `Book`

O entitate deodată, build + rulare între ele. Nu ai de inventat nimic.

Pas	Entitate	Ce faci	Detaliile
1	<code>Course</code>	T0.5 → T0.6 → T1, în ordinea asta	secțiunile T0.5, T0.6, T1 de mai jos
2	<code>Enrolment</code>	la fel	idem
3	<code>User</code>	doar T0.5 — DTO-uri <code>record</code> + <code>UserMapper</code>	T0.5

`User` se oprește la T0.5 intenționat. `ITextMapper<User>.FromText` ar trebui să decidă singur dacă naște un `Student`, un `Teacher` sau un `Admin` — iar asta nu se poate face cu polimorfism. Repository-ul lui vine în Faza 2, cu Factory Method. Explicația lungă e la T1, „De ce `User` nu intră acum”.

Un lucru de observat, nu de făcut: copiind `Book`, aplici deja un **Strategy** — `Repository<T>` ține un `ITextMapper<T>` fără să știe care e, exact ca `Raport` cu `IExportStrategie` în `ex3`. Îl aplici, nu îl proiectezi. Pe ăla îl proiectezi tu în Faza 2.

O îmbunătățire față de model: `BookRepository` își fixează singur calea (`Path.Combine("..", "..", "..", "Data", "books.txt")`). La `Course` și `Enrolment`, dă calea **din afară**, de la cel care construiește repo-ul (`ViewStudent`). Configurația care vine din exterior e forma pe care o vei vedea peste câteva luni ca fișier de configurare și connection string — și e singurul lucru care ar face codul ăsta testabil.

Faza 2 — pattern-urile, în ordine

Se deschid pe rând. Ordinea e impusă de cod, nu de programa lecțiilor.

Task	Pattern	Unde aterizează	De ce acolo
T3	Factory Method	<code>UserService.ReadUsers</code> (<code>switch</code> pe tip) și <code>CreateUser</code> (lanț as + <code>if</code>)	poarta — fără el <code>User</code> nu intră niciodată în <code>Repository<T></code>
T1-User	Strategy	<code>UserTextMapper</code> , folosind fabricile de la T3	abia acum moare bug-ul cu profesorul care nu se recitește
T1b	Strategy, proiectat de tine	validările din <code>Teacher</code> / <code>Admin</code>	<code>Salary</code> și <code>Password</code> avertizează cu <code>Console.WriteLine</code> și atribuie oricum , în timp ce <code>WorkHours</code> și <code>Age</code> aruncă. Aceeși regulă, copiată în două clase. Țasta e <code>ex5</code> mutat aici
T2	Observer	serviciile devin sursă; <code>AutoSave</code> + <code>JurnalAudit</code> ascultă	<code>Save()</code> există în toate serviciile și nu e chemat niciodată
T8	Decorator	<i>de stabilit</i>	îl fixăm la lecția 4

Ordinea are un motiv dur: `Teacher.ToText` scrie 8 câmpuri și nu scrie parola, iar `Teacher(string)` citește `cuv[8]`. În clipa în care ceva chiar salvează, `users.txt` se rescrie stricat și aplicația nu mai pornește. De-aia Observer (salvarea automată) vine **după** ce `User` trece printr-un mapper, nu înainte.

Restul — Singleton, Builder, Adapter, State — rămân în „Ce urmează”, la finalul fișierului.

Când citești T0.5, T0.6 și T1: referințele `file:line` care arată spre `Books/` descriu codul **de dinainte** — fișierele alea sunt deja rescrise, uită-te direct în cod. Pentru `Course` și `Enrolment`

descrierile sunt corecte; acolo n-am atins nimic.

T0 — Reparații, fără niciun pattern

T0.0–T0.4: ÎNCHISE  (`79abec3` , 2026-09-08) — verificate prin rulare, toate cinci. Rămân aici ca istoric; nu mai ai nimic de făcut în ele.

T0.5 și T0.6 sunt vii — sunt pașii din Faza 1, aplicați acum pe `Course` , `Enrolment` și `User` .

T0.0 — Proiectul nu compilează

Commit-ul `999504c` nu trece de build:

```
Users/Dtos/AdminUpdateRequest.cs(1,49): error CS0234:  
The type or namespace name 'Admins' does not exist in the namespace  
'stefan_academy_vanilla_charp.Users.Models'
```

Ai mutat `Admin` , `Student` și `Teacher` din `Users/Models/<X>s/Models/` direct în `Users/Models/` — mutare bună, curăță „Models” dublat din namespace. Dar ai actualizat `using` -urile doar în fișierele pe care le aveai deschise. `AdminUpdateRequest.cs` a rămas cu cel vechi.

Aceeași greșeală ca la `design-patterns` , runda 4: o redenumire sau o mutare nu e o editare într-un fișier, e o operație peste tot codul care se referă la el. În Visual Studio, mută clasa cu drag & drop în Solution Explorer — actualizează namespace-ul și `using` -urile singur.

Regula, de acum înainte: `Ctrl+Shift+B` înainte de fiecare commit. Nu „am terminat de scris”, ci „am dat build și trece”.

Gata când: `Build succeeded` , 0 erori.

T0.1 — „Vezi cursurile” crapă

`ViewStudent.cs:126` → `CourseService.cs:29–39`

Rulează aplicația, loghează-te ca student, meniul `2` → `1` . Primești:

```
Unhandled exception. System.NullReferenceException  
at ViewStudent.AfisareCursuri() in ViewStudent.cs:line 131
```

Citește cu atenție ce trimiți și ce se caută:

```
courseService.GetCourseListByEnrolmentId(enrolmentService.GetEnrolmentIdByStudentId(loggedUser.Id))
```

`GetEnrolmentIdByStudentId` întoarce ID-uri de **înrolare**. `GetCourseListByEnrolmentId` le dă mai departe lui `FindById`, care caută printre ID-uri de **curs**. Nu se potrivesc niciodată, deci `FindById` întoarce `null` de fiecare dată, iar lista se umple cu `null`-uri.

Îți lipsește un pas. O înrolare leagă un student de un curs — deci de la înrolare la curs se ajunge prin `CourseId`, nu prin `Id`.

Gata când: un student înscris la cursuri le vede afișate, iar unul neînscris primește „Nu sunteți înscris la niciun curs”.

T0.2 — „Modifică o carte” face cartea să dispară

`ViewStudent.cs:193`

Rulează: meniul 1 → 1 (vezi că ai cărți) → 3 (modifici una) → 1 din nou. Cartea a dispărut.

```
BookUpdateRequest request = new BookUpdateRequest(book.Id, nume, DateTime.Now);
```

Uită-te la semnătura constructorului. Primul parametru **nu** e ce crezi tu că e. Apoi urmărește ce face `BookService.UpdateBook` cu el.

Ambele sunt `Guid`, deci compilatorul n-are ce să-ți spună. Asta e o clasă de greșeală care se va repeta — o rezolvăm din rădăcină la T5.

Gata când: după modificare cartea rămâne a ta, cu numele nou, și data la care ai adăugat-o inițial nu se schimbă.

T0.3 — Data înscrierii se pierde

`Enrolment.cs:25-28`

```
public Enrolment(Guid studentId, Guid courseId, DateTime createdAt) {  
    StudentId = studentId;  
    CourseId = courseId;  
}
```

Citește constructorul până la capăt și numără parametrii folosiți.

Gata când: o înrolare creată cu o dată anume o păstrează.

T0.4 — CreateUser construiește mereu un User gol

Users/Services/UserService.cs:81-88

În 999504c ai scos lanțul de `if` pe tip din `CreateUser` și l-ai înlocuit cu:

```
User newUser = new();  
newUser.Create(request);
```

Instinctul e corect — lanțul ăla de `as` + `if` chiar trebuia să dispară, și l-ai înlocuit cu polimorfism, care e unealta pe care o ai. Dar uită-te ce tip are `newUser` pe prima linie.

`Create` e `virtual`, deci apelul se duce la implementarea **tipului real al obiectului**. Iar obiectul e un `User` — l-ai creat cu `new User()`. Deci se cheamă `User.Create`, niciodată `Teacher.Create` sau `Admin.Create`. Overrides-urile pe care le-ai scris în `Teacher.cs:98` și `Admin.cs` nu se execută niciodată.

Consecințe, în ordinea în care le vei vedea: 1. Creezi un profesor → salariul, orele și parola se pierd în tăcere. 2. `ToText` scrie linia ca `USER,...`. 3. La următoarea pornire, `switch`-ul din `ReadUsers` cade pe `default` → `ArgumentException("Eroare in fisierul de citire!!!")` și aplicația nu mai pornește.

Aici e lecția, și merită ținută minte: **polimorfismul alege ce metodă se execută pe un obiect care există deja. Nu poate alege ce clasă să construiască** — pe aia o decizi tu, înainte, când scrii `new`.

„Cine decide ce clasă construim” e o problemă separată, și are un pattern al ei: **Factory Method**, T3.

Deocamdată: pune la loc varianta care funcționa (lanțul de `if`), ca proiectul să fie corect. O scoatem definitiv la T3, cu unealta potrivită. Nu e un pas înapoi — e ordinea corectă: întâi funcționează, apoi e frumos.

Gata când: creezi un profesor, îl salvezi, repornești aplicația, și profesorul e tot profesor, cu salariu și parolă.

T0.5 — DTO-urile moștenesc entitățile

Book e deja făcut, în cod, ca model de urmat. Deschide `Books/Dtos/`, `Books/Mappers/` și `Books/Services/BookService.cs` și compară-le cu `Courses/` și `Enrolments/` — ai varianta veche și cea nouă una lângă alta. Tu faci `Course` și `Enrolment` după același tipar, apoi `Users` la urmă.

`Books/Dtos/BookCreateRequest.cs:10` · `BookUpdateRequest.cs:10` ·

`Courses/Dtos/CourseCreateRequest.cs:10` · `CourseUpdateRequest.cs:10` ·

`Enrolments/Dtos/EnrolmentCreateRequest.cs:10` · `EnrolmentUpdateRequest.cs:10`

Șase clase de request încep la fel:

```
public class BookUpdateRequest : Book
```

Un request care moștenește entitatea **este** entitatea: are `Id`-ul ei (un `Guid.NewGuid()` generat și aruncat la fiecare cerere), are toate câmpurile ei, trece prin validările ei și poate fi pasat oriunde se așteaptă un `Book`. Nu mai există nicio graniță între „ce cere clientul” și „ce ține baza de date”.

Asta e cauza bug-ului pe care l-ai reparat la T0.2. Uită-te la linia veche:

```
BookUpdateRequest request = new BookUpdateRequest(book.Id, nume, DateTime.Now);
```

`StudentId` exista în request **doar** pentru că e moștenit din `Book`. Nimeni n-a decis vreodată „la modificarea unei cărți, clientul are voie să schimbe proprietarul” — a venit gratis odată cu `: Book`. Tu ai reparat valoarea trimisă. Câmpul e tot acolo.

Pe `Users` ai făcut exact pe dos: `UserCreateRequest` (143 linii) și `UserUpdateRequest` (142 linii) sunt validarea din `User` copiată cu mâna. Aceeași întrebare, două răspunsuri opuse, în același proiect. Semnul că sunt două copii, nu una: în `Teacher`, setterul `Salary` doar scrie un mesaj și atribuie oricum, iar `WorkHours` aruncă excepție — și exact aceeași asimetrie e copiată în `TeacherCreateRequest`. Două locuri care trebuie ținute în pas la fiecare modificare.

Ce construiești

1. DTO-urile devin `record` -uri. Un `record` e o clasă de date: câmpuri needitabile după construcție, egalitate pe valoare, un `ToString` folositor la depanare — și, cel mai important aici, compilatorul îți interzice moștenirea greșită:

```
error CS8864: Records may only inherit from object or another record
```

Adică decizia de design nu mai depinde de disciplina ta: `record ... : Book` **nu compilează**. Regula se aplică singură.

`Books/Dtos/BookCreateRequest.cs` — 18 linii devin una:

```
namespace stefan_academy_vanilla_charp.Books.Dtos
{
    public record BookCreateRequest(Guid StudentId, string BookName, DateTime CreatedAt);
}
```

`Books/Dtos/BookUpdateRequest.cs` — și aici e partea importantă. Nu copia câmpurile vechi: întreabă-te **ce are voie să schimbe o modificare de carte**. Proprietarul, nu. Data adăugării, nu (ai stabilit-o la T0.2). Rămâne numele:

```
namespace stefan_academy_vanilla_charp.Books.Dtos
{
    public record BookUpdateRequest(string BookName);
}
```

`BookCreateResponse` / `BookUpdateResponse`, la fel:

```
public record BookCreateResponse(Guid Id, Guid StudentId, string BookName, DateTime CreatedAt);
public record BookUpdateResponse(Guid Id, string BookName, DateTime CreatedAt);
```

2. Mapperele își primesc folderul lor. Traducerea DTO ↔ entitate nu e treaba serviciului — serviciul ține lista și regulile de business. Azi mapperle stăteau într-o secțiune `//Mappers` în mijlocul lui `BookService`, și nici acolo nu trecea totul prin ea: `CreateBook` chema mapperul, dar `UpdateBook` atribuia câmpurile cu mâna.

Structura ta pe funcționalități are deja `Models` , `Dtos` , `Services` . Mai lipsea unul:

```
Books/  
  Models/  
  Dtos/  
  Mappers/      <-- nou  
  Repositories/ <-- nou, T0.6  
  Services/
```

`Books/Mappers/BookMapper.cs` :

```

using stefan_academy_vanilla_charp.Books.Dtos;
using stefan_academy_vanilla_charp.Books.Models;

namespace stefan_academy_vanilla_charp.Books.Mappers
{
    public static class BookMapper
    {
        public static Book ToBook(BookCreateRequest request)
        {
            return new Book(request.StudentId, request.BookName, request.CreatedAt);
        }

        public static void ApplyUpdate(Book book, BookUpdateRequest request)
        {
            book.BookName = request.BookName;
        }

        public static BookCreateResponse ToCreateResponse(Book book)
        {
            return new BookCreateResponse(book.Id, book.StudentId, book.BookName, book.CreatedAt);
        }

        public static BookUpdateResponse ToUpdateResponse(Book book)
        {
            return new BookUpdateResponse(book.Id, book.BookName, book.CreatedAt);
        }
    }
}

```

Numele s-au scurtat: în interiorul unei clase numite `BookMapper`, `BookCreateRequestToBook` se bâlbâie. `BookMapper.ToBook(request)` se citește dintr-o bucată.

De ce `static` aici, când la T4 o să spunem că `static` e răspunsul leneș? Fiindcă întrebarea nu e „static sau nu”, ci „**are obiectul ăsta stare?**”. `BookService` are: ține lista de cărți. Două instanțe = două liste care nu se văd (exact bug-ul de la T4). `BookMapper` nu ține nimic — primește un obiect, întoarce altul, nu-și amintește nimic între apeluri. Nu există „două mappere diferite”, deci nici motiv să instanțiezi unul. ăsta e cazul în care `static` e răspunsul corect, nu scurtătura.

`ApplyUpdate` e metoda care lipsea cu totul. Are un singur rol: **este singurul loc din proiect care știe ce câmpuri ale unei cărți poate atinge o cerere de modificare**. Cât timp regula e scrisă într-o singură metodă, nu poate fi încălcată din greșeală în altă parte.

3. `UpdateBook` nu mai atinge niciun câmp. Serviciul găsește, verifică, deleagă:

```
public BookUpdateResponse UpdateBook(Guid id, BookUpdateRequest request)
{
    Book book = FindById(id);
    if (book == null)
    {
        throw new ArgumentException("Cartea nu exista in baza de date");
    }

    BookMapper.ApplyUpdate(book, request);

    return BookMapper.ToUpdateResponse(book);
}
```

Regula, de acum înainte: un serviciu nu scrie niciodată direct într-un câmp de entitate. Dacă te prinzi scriind `x.Ceva = request.Ceva` în afara unui mapper, îți lipsește un mapper. `BookService` nu mai are nicio linie de genul ăsta — verifică singur.

4. Apelul din `ViewStudent.cs:194` devine:

```
BookUpdateRequest request = new BookUpdateRequest(ume);
```

Uită-te bine la linia asta. **Nu mai există niciun `Guid` de pus pe poziția greșită**. Bug-ul de la T0.2 nu mai e reparat — e imposibil de scris. Asta e diferența dintre a corecta o linie și a închide o clasă de greșeli.

Ce NU face `record` -ul

Să nu rămâi cu impresia că rezolvă tot. `EnrolmentCreateRequest(Guid StudentId, Guid CourseId, DateTime CreatedAt)` are în continuare doi `Guid` unul lângă altul, iar dacă îi inversezi compilatorul tace la fel de mult ca înainte. Împotriva **ăia** ai două unelte: argumente numite la apel —

```
new EnrolmentCreateRequest(studentId: loggedUser.Id, courseId: course.Id, createdAt: DateTime
```

— și, mai târziu, **T5 (Builder)**. `record` -ul rezolvă altceva: imutabilitatea, boilerplate-ul și moștenirea greșită.

Ordinea

Book e gata (fă `git show` pe commit-ul ăsta ca să vezi exact ce s-a schimbat și ce **nu** s-a schimbat). Urmează `Course` și `Enrolment`, identice ca formă, cu build între ele.

`UserCreateRequest` și `UserUpdateRequest` le lași **la urmă**: acolo nu e doar o moștenire de șters, ci 285 de linii de validare de mutat înapoi în `User`, `Teacher` și `Admin`. Le facem separat, după T1.

Gata când: 1. `grep -rn "Request : \|Response : " Books Courses Enrolments` nu mai găsește nimic (pe `Users`, `record` -urile pot moșteni `record` -uri, e în regulă). Pe `Books` e deja curat. 2. ✓ Apelul de modificare a unei cărți nu mai conține niciun `Guid` — `ViewStudent.cs:194`. 3. Există `Courses/Mappers/CourseMapper.cs` și `Enrolments/Mappers/EnrolmentMapper.cs`, iar `UpdateCourse` și `UpdateEnrolment` nu mai au nicio atribuire de câmp în corpul lor — cum n-are nici `UpdateBook`. 4. Build verde, și scenariul de la T0.2 rulat din nou: modifici o carte, rămâne a ta, cu data inițială.

T0.6 — Serviciul face trei meserii deodată

`Books/` e deja făcut, ca model. Tu faci `Courses`, `Enrolments` și `Users`.

De ce

Uită-te ce era în `BookService` înainte:

- ținea lista în memorie — `private readonly List<Book> books`
- vorbea cu fișierul — `ReadBooks`, `Save`, `BooksListToString`, `Path.Combine(...)`
- căuta prin listă — `FindById`, `GetBook`, `GetBooksByStudentId`
- aplica regulile — „cartea există?”, „e deja în bază?”

Patru meserii într-o clasă de 152 de linii. Semnul că e prea mult nu e numărul de linii, ci **numărul de motive pentru care ai deschide fișierul**: dacă mâine treci de la `books.txt` la o bază de date, ai de umblat în serviciu. Dacă mâine schimbi regula „cine poate modifica o carte”, tot în serviciu. Două schimbări care n-au nicio legătură una cu alta ajung în același loc și se calcă pe picioare.

Repartizarea corectă:

Cine	Ce știe	Ce NU știe
<code>BookRepository</code>	unde stau cărțile și cum se caută printre ele	ce înseamnă „carte inexistentă” pentru aplicație
<code>BookService</code>	regulile: ce e valid, ce aruncă excepție	dacă datele vin din fișier, din memorie sau din SQL
<code>BookMapper</code>	cum se traduce DTO ↔ entitate	orice altceva

Ce construiești

`Books/Repositories/BookRepository.cs` primește lista, fișierul și căutările:

```
public Book FindById(Guid id)
public Book FindByStudentAndName(Guid studentId, string bookName)
public List<Book> FindByStudentId(Guid studentId)
public void Add(Book book)
public void Remove(Book book)
public void Save()
private void Read()
```

`BookService` rămâne doar cu regulile — 65 de linii, și fiecare metodă are aceeași formă: **întreabă repo-ul, verifică existența, deleagă**:

```
public BookUpdateResponse UpdateBook(Guid id, BookUpdateRequest request)
{
    Book book = repository.FindById(id);
    if (book == null)
    {
        throw new ArgumentException("Cartea nu exista in baza de date");
    }

    BookMapper.ApplyUpdate(book, request);

    return BookMapper.ToUpdateResponse(book);
}
```

În tot serviciul nu mai există niciun `List<>`, niciun `StreamReader` și niciun `Path.Combine`. Verifică singur cu `grep`.

Injectia: repo-ul se PRIMEȘTE, nu se fabrică

```
public class BookService
{
    private readonly BookRepository repository;

    public BookService(BookRepository repository)
    {
        this.repository = repository;
    }
}
```

Diferența față de `private readonly BookRepository repository = new();` pare cosmetică. Nu e.

Cât timp o clasă își face singură dependențele cu `new`, **nimeni din afară nu poate schimba cu ce lucrează ea**. Nu poți să-i dai un repo de test cu 3 cărți fixe. Nu poți să-i dai un repo care citește din altă parte. Și, cel mai important pentru bug-ul pe care îl ai: nu poți să-i dai **același** repo pe care îl are altcineva — fiecare `new` face o copie proprie.

Uită-te acum la `ViewStudent.cs:15`:

```
private BookService bookService = new(new BookRepository());
```

Decizia „ce repo folosește serviciul” s-a mutat din serviciu în cel care îl construiește. E doar jumătate de pas — `ViewStudent` tot fabrică, în loc să primească — dar e jumătatea care contează, fiindcă abia acum **există** un loc unde poți decide altfel. A doua jumătate e **T4**, unde `ViewStudent` primește serviciile prin constructor, cum primește deja `User`.

O schimbare de comportament, intenționată

`DeleteBook` ștergea în tăcere dacă nu găsea nimic. Acum aruncă, la fel ca `UpdateBook`:


```
Book book = repository.FindById(id);
if (book == null)
{
    throw new ArgumentException("Cartea nu exista in baza de date");
}
```

Din meniu nu se vede nicio diferență, fiindcă `ViewStudent` verifică deja cu `GetBook` înainte să cheme. Dar regula „ce se întâmplă când ceva nu există” trebuie să fie **aceeași în tot serviciul**, altfel apelantul trebuie să țină minte, metodă cu metodă, care aruncă și care tace.

De gândit

În `CreateBook` a rămas:

```
if (repository.FindById(newBook.Id) != null)
{
    throw new ArgumentException("Cartea se afla deja in baza de date");
}
```

Uită-te de unde vine `newBook.Id` și spune-mi: **poate fi vreodată adevărată condiția asta?** Dacă nu, ce voiai de fapt să verifici acolo? (Aceeași întrebare e valabilă în `CourseService` și `EnrolmentService` — e copiată în toate trei.)

Gata când: 1. Există `Courses/Repositories/CourseRepository.cs` și `Enrolments/Repositories/EnrolmentRepository.cs`. 2. `grep -rn "StreamReader\|StreamWriter\|Path.Combine" Books Courses Enrolments Users` găsește numai fișiere din `Repositories/`. 3. Niciun serviciu nu mai declară `List<...>`. 4. Serviciile primesc repository prin constructor. Niciun `new SomethingRepository()` în interiorul unui serviciu. 5. Build verde + fluxurile de cărți și cursuri rulate din meniu.

T1 — Strategy: persistența într-un singur loc

Pattern: Strategy (lecția 1).

Book e făcut complet, în cod, ca model de urmat. Deschide `Common/` și `Books/` și ai toată forma sub ochi. Tu faci `Course` și `Enrolment`; `User` așteaptă T3 și mai jos scrie de ce.

De ce

Trei probleme, aceeași cauză.

1. Scrierea și citirea stau în fișiere diferite și au divergat. `Teacher.ToText`

(`Users/Models/Teacher.cs:82`) scrie 8 câmpuri și uită parola. `Teacher(string text)`

(`Users/Models/Teacher.cs:28`) citește `cuv[8]`. Am rulat testul: profesor salvat pe ultima linie → se recitește cu parola goală și nu se mai poate loga; profesor oriunde altundeva →

`IndexOutOfRangeException` la pornire.

2. Modelele știu că sunt salvate într-un fișier. `ToText(int cnt, int size)` — modelul primește poziția lui în listă ca să decidă dacă pune `\n` la final (`User.cs:147`, `Student.cs:19`, `Teacher.cs:82`, `Admin.cs:56`). Un `Teacher` n-are de ce să știe câți alți useri există.

3. Aceeași buclă, scrisă de patru ori: `UserService.cs:123 + :162 + :172`, `CourseService.cs:108 + :149 + :166`, `EnrolmentService.cs:123 + :162 + :179`. Plus `Path.Combine("..", "..", "..", "Data", ...)` peste tot. (`Books` e deja curățat — de-aia nu mai apare în listă.)

Ce construiești

1. Promisiunea: `Common/IEntity.cs`

```
public interface IEntity
{
    Guid Id { get; }
}
```

Cele patru entități au fiecare `public Guid Id { get; set; }`, dar pe patru tipuri fără nicio legătură între ele. Compilatorul nu vede acolo un tipar, vede patru coincidențe — deci `FindById` nu poate urca într-o clasă generică. **Codul generic are nevoie de o promisiune despre `T`, iar constrângerea `where T : class, IEntity` ESTE promisiunea.** Fără ea, `T` e `object` și nu poți face nimic cu el.

Costul: fiecare entitate primește `: IEntity` și atât — membrul îl are deja. `Book` e făcut; mai sunt `Course`, `Enrolment`, `User`.

2. Strategia: `Common/ITextMapper.cs`

```
public interface ITextMapper<T>
{
    string ToText(T item);
    T FromText(string text);
}
```

(Era în `Users/Models/` — un contract generic n-avea ce căuta acolo. A fost mutat în `Common/`.)

Implementarea, `Books/Mappers/BookTextMapper.cs`, cu cele două metode **una sub alta**:

```
public string ToText(Book item)
{
    return item.Id + "," + item.StudentId + "," + item.BookName + "," + item.CreatedAt.ToString();
}

public Book FromText(string text)
{
    string[] cuv = text.Split(',');

    Book book = new Book(Guid.Parse(cuv[1]), cuv[2], DateTime.Parse(cuv[3]));
    book.Id = Guid.Parse(cuv[0]);

    return book;
}
```

Ăsta e miezul task-ului: bug-ul de la punctul 1 devine greu de făcut, fiindcă vezi ambele capete deodată. Adaugi un câmp în `ToText` și `FromText` e chiar sub el.

3. Contextul: Common/Repository.cs

```
public class Repository<T> where T : class, IEntity
{
    private readonly List<T> items = new();
    private readonly ITextMapper<T> mapper;
    private readonly string path;

    public Repository(ITextMapper<T> mapper, string path)
    {
        this.mapper = mapper;
        this.path = path;
        Read();
    }

    protected List<T> Items { get { return items; } }

    public T FindById(Guid id)
    public void Add(T item)
    public void Remove(T item)
    public void Save()
    private void Read()
}
```

Repository nu știe ce e un **Book**. Știe să citească linii, să ceară mapperului să le traducă, și invers. Observă cine pune acum `\n` între linii: **Save**, în buclă. **Nu modelul**. Asta rezolvă punctul 2.

4. Moștenirea: BookRepository : Repository<Book>

```
public class BookRepository : Repository<Book>
{
    public BookRepository()
        : base(new BookTextMapper(), Path.Combine("..", "..", "..", "Data", "books.txt")) { }

    public Book FindByStudentAndName(Guid studentId, string bookName)
    public List<Book> FindByStudentId(Guid studentId)
}
```

Din 102 linii au rămas 30, și toate cele 30 sunt despre cărți. `FindById`, `Add`, `Remove`, `Save`, `Read` vin din bază.

Moștenirea e corectă aici pentru că `BookRepository` **adaugă** și nu **ascunde** nimic. Regula: moștenești ca să adaugi. În clipa în care o subclasă trebuie să facă ilegală o metodă moștenită, moștenirea era unealta greșită.

Capcana pe care am ocolit-o

Varianta la care ajunge oricine prima dată:

```
public abstract class Repository<T>
{
    protected abstract T FromText(string line);
    protected abstract string ToText(T item);
}
```

Compilează, e mai puțin de scris, și **anulează T0.6**. Pentru că atunci `BookRepository` conține iar două lucruri fără legătură: interogările despre cărți și formatul în care cărțile ajung pe disc. Ca să treci de la `books.txt` la JSON, ai deschide fișierul care ține căutările.

Ăsta e chiar contrastul dintre **Template Method** (subclasa completează golurile din bază) și **Strategy** (bazei i se dă o piesă, din afară). Diferența practică vine la **T6, Adapter**: cu mapper injectat, pui `System.Text.Json` în spatele lui `ITextMapper` și nu atingi niciun repository. Cu metode abstracte, T6 cere rescriere.

De ce `User` nu intră acum

`Repository<User>` ar trebui să transforme o linie în `Student`, `Teacher` sau `Admin`, după prefix. Un `ITextMapper<User>.FromText` poate face asta cu un `switch` — dar ăla e chiar lanțul de la T0.4, mutat în alt fișier. „Cine decide ce clasă se construiește” e **T3, Factory Method**.

Deci ordinea e: `Course` și `Enrolment` acum, `User` la T3. Dacă te blochezi la `User`, nu e vina ta — e task-ul următor.

Ce atingi

- `Course` și `Enrolment` primesc `: IEntity`.

- Scrii `CourseTextMapper` , `EnrolmentTextMapper` , apoi `CourseRepository : Repository<Course>` și `EnrolmentRepository : Repository<Enrolment>` .
- Scoți `ReadX()` , `XListToString()` , `Save()` din `CourseService` și `EnrolmentService` ; ele primesc repo-ul prin constructor, ca `BookService` .
- Ștergi constructorii `Course(string text)` și `Enrolment(string text)` — treaba aia e acum a mapperului. (`Book(string text)` e deja șters.)

Ce NU atingi

`ViewLogIn` , DTO-urile, validările din setteri. În `ViewStudent` schimbi doar linia care construiește serviciile.

Gata când

- `Data/*.txt` rămân neschimbate ca format — scopul e ca nimic din afară să nu observe refactorul.
 - Adaugi manual o linie **în mijlocul** lui `courses.txt` , pornești, salvezi, pornești din nou: pornește și cursul e acolo. (Pe `books.txt` testul ăsta trece deja — l-am rulat: 8 linii scrise, 8 recitite, cu aceleași `Id` -uri și date.)
 - `grep -rn "StreamReader\|StreamWriter\|Path.Combine" Books Courses Enrolments` nu mai găsește nimic în afara lui `Repositories/` .
 - Niciun model din `Books` , `Courses` , `Enrolments` nu mai conține „txt”, `\n` sau vreo virgulă de separator.
-

T1b — Strategy: o regulă, un singur loc

Pattern: Strategy (lecția 1) — de data asta îl **proiectezi tu**, nu îl copiezi. **Depinde de:** T0.5 pe **Users**.

De ce

Deschide **Users/Models/Teacher.cs** la linia 37 și citește setterul lui **Salary**:

```
set
{
    if (value < 4257)
    {
        Console.WriteLine("Salariul trebuie sa fie cel putin minim pe economie");
    }
    salary = value;
}
```

Uită-te la ce face: **spune că valoarea e greșită, și o pune oricum**. Un profesor cu salariul 100 intră în sistem, cu un mesaj pe ecran pe care nimeni nu-l citește. La fel **Password**, la **Teacher.cs:50**.

Iar acum uită-te la vecinii lor din aceeași clasă — **WorkHours** aruncă **ArgumentException**, și la fel **Age** din **User**. Deci ai patru reguli și două comportamente: două opresc, două fac teatru. Nimic din cod nu-ți spune care e intenția.

A doua problemă e mai urâtă. Caută în tot proiectul textul „Salariul trebuie sa fie cel putin minim pe economie”:

Users/Models/Teacher.cs:44	Users/Dtos/TeacherCreateRequest.cs:36
Users/Models/Admin.cs:36	Users/Dtos/TeacherUpdateRequest.cs:35
	Users/Dtos/AdminCreateRequest.cs:33
	Users/Dtos/AdminUpdateRequest.cs:23

Șase copii ale aceleiași reguli. Regula parolei: tot șase. Regula orelor: trei. Cincisprezece bucăți de cod care trebuie ținute în pas cu mâna. Prima dată când schimbi pragul salariului, îl schimbi în șase locuri — sau, mai probabil, în trei.

T0.5 rezolvă deja jumătate: când request-urile devin `record` -uri, cele patru copii din `Dtos/` dispar prin construcție, fiindcă un `record` n-are settere în care să le pui. De-aia task-ul ăsta vine **după**. Rămân cele două din modele — și acolo intră Strategy.

Ce construiești

Contractul, în `Users/Validari/` :

```
public interface IRegulaValidare<T>
{
    string Nume { get; }
    bool EsteValida(T valoare);
    string Mesaj { get; }
}
```

Implementările: `SalariuMinim` , `ParolaPuternica` , `OreDeLucru` , `VarstaMinima` . Fiecare știe o **singură** regulă și pragul ei, primit prin constructor — nu hardcodat.

`Teacher` și `Admin` nu mai conțin nicio condiție: primesc regulile din afară și le întreabă, exact cum `CampParola` din `ex5` ține o `IPoliticaParola` și o întreabă `EsteValida` .

Decizia pe care o iei tu

Reparația minimă aici nu e un pattern, e un `throw` . Dacă cele patru reguli rămân fixe și identice pentru toți, transformi cele două `Console.WriteLine` în `throw` și ai terminat în două minute — și ăsta ar fi răspunsul onest.

Strategy își merită locul doar dacă regulile **variază**: parolă mai strictă pentru admin decât pentru profesor, praguri de salariu diferite pe rol, poate reguli diferite pentru un cont de test.

Deci înainte să scrii o linie, răspunde în scris, două rânduri: **variază regulile sau nu?** Dacă răspunsul e „nu”, fă `throw` -ul și treci la T2 — și asta e un rezultat corect al task-ului, nu un eșec. Recitește „Când NU îl folosești” din lecția 1.

Ce atingi

`Users/Models/Teacher.cs` , `Users/Models/Admin.cs` , `Users/Models/User.cs` (setterele), plus clasele noi din `Users/Validari/` . Cine construiește userii le dă regulile.

Ce NU atingi

Repository-urile, mapperele, serviciile. Nimic din `Books`, `Courses`, `Enrolments`.

Gata când

- `grep -rn "minim pe economie" --include=*.cs .` întoarce **un singur** rezultat, în clasa regulii.
 - O valoare invalidă **nu mai intră** în obiect — nici pentru salariu, nici pentru parolă. Verifică prin rulare: creează un profesor cu salariul 100 și uită-te dacă ajunge în `users.txt`.
 - Cele patru reguli se comportă la fel: sau toate opresc, sau toate raportează. Fără două opriri și două mesaje.
 - În `Teacher` și `Admin` nu mai există niciun `if` de validare.
-

T2 — Observer: salvarea se întâmplă singură

Pattern: Observer (lecția 2). **Depinde de:** T1, inclusiv pe `User`. **Ultimul task din Faza 2** — vezi avertismentul de mai jos, nu-l lua înaintea rândului.

De ce

Rulează aplicația, adaugă o carte, ieși, pornește din nou. Cartea nu mai e.

`Save()` e definit în patru locuri — `Common/Repository.cs:43`, `CourseService.cs:166`, `UserService.cs:172`, `EnrolmentService.cs:179` — și chemat în **zero**. Verifică singur: `grep -rn "\.Save()" --include=*.cs` . nu întoarce nimic. Aplicația spune „adaugata cu succes” și pierde tot la ieșire.

Soluția evidentă e să pui `Save()` la finalul fiecărei metode care schimbă ceva. Merge — și e exact capcana din lecția 1: te obligă să-ți amintești, de fiecare dată, în fiecare metodă nouă. Prima dată când uiți, pierzi date fără niciun mesaj de eroare.

Aici sursa trebuie doar să **anunțe** că s-a schimbat ceva. Cine reacționează și cum nu e treaba ei.

Ce construiești

```
public interface IObservatorDate
{
    void DateModificate(string sursa);
}
```

Serviciile devin Subject: la fiecare `Create`, `Update`, `Delete` anunță observatorii.

În `BookService` sunt exact trei puncte în care starea se schimbă, deci exact trei locuri de unde se anunță:

```
repository.Add(newBook);           // :36
BookMapper.ApplyUpdate(book, request); // :49
repository.Remove(book);           // :62
```

Doi observatori:

Clasă	Reacția
<code>AutoSave</code>	cheamă <code>Save()</code> pe repository
<code>JurnalAudit</code>	scrie ce s-a schimbat și când

De ce doi, și nu doar `AutoSave`

Pentru că altfel **nu e Observer**, și ți-aș încălca chiar regula pe care ți-am dat-o. Recitește „Când NU îl folosești” din lecția 2: *ai un singur ascultător și va rămâne unul — un apel direct e mai clar*. Cu un singur `AutoSave`, varianta onestă ar fi `repository.Save()` scris direct în serviciu.

`JurnalAudit` nu e umplutură: aplicația n-are azi nicio urmă a ce s-a întâmplat — cine a șters o carte, cine s-a înscris la ce curs, când. Doi ascultători independenți la același eveniment, care nu știu unul de altul: **asta** e situația pe care Observer o rezolvă.

Modelul e `ex8` din repo-ul de design patterns: `Cont.Depune / Retrage` schimbă soldul și cheamă `NotificaObservatori()`, iar `Cont` habar n-are că există `AlertaSoldMic`. Aici, `BookService` nu va ști că există `AutoSave`.

Ce atingi

Cele 4 servicii (devin Subject), plus clasele noi. `Program.cs` sau `ViewLogIn` leagă observatorii la servicii.

Ce NU atingi

Mapper-ele și `Repository` din T1.

Gata când

- Adaugi o carte, ieși, pornești din nou: cartea e acolo.
- Te înscrii la un curs, ieși, pornești din nou: înscrierea e acolo.
- Ștergi `AutoSave` de la înregistrare și totul funcționează la fel, doar că nu se mai salvează. Dacă trebuie să modifichi vreun serviciu ca să faci asta, Observer-ul nu e corect.
- În servicii nu apare cuvântul `Save`.

Atenție — capcană din lecția 2: `InscriereCurs` (`ViewStudent.cs:237`) adaugă direct în lista expusă de serviciu, sărind peste `CreateEnrolment` . Nicio notificare nu se va declanșa de acolo. Repară și asta.

De ce task-ul ăsta e ULTIMUL

Nu e o preferință de programă, e o chestiune de date pierdute.

`Teacher.ToText` scrie 8 câmpuri și **nu scrie parola**; `Teacher(string text)` citește `cuv[8]` . Azi nu se vede, fiindcă nimic nu salvează — `users.txt` are 7 studenți și un admin, exact așa cum au fost scriși de mână.

În clipa în care `AutoSave` chiar funcționează, fișierul se rescrie în formatul stricat. Profesor pe ultima linie → se recitește cu parola goală și nu se mai poate loga. Profesor oriunde altundeva →

`IndexOutOfRangeException` la pornire, aplicația nu mai boot-ează.

Deci: întâi T3 (Factory Method) și T1 pe `User` , ca scrierea și citirea să ajungă una sub alta în același mapper. Abia apoi Observer. Task-ul ăsta e cel care armează bomba; asigură-te că ai dezamorsat-o înainte.

Ce urmează — după Faza 2

Pattern-uri pe care nu le-am făcut încă la lecție. Fiecare are deja locul lui în cod, te așteaptă acolo.

Task	Pattern	Unde aterizează
T4	Singleton (și de ce de obicei NU)	<code>ViewLogIn</code> și <code>ViewStudent</code> construiesc fiecare propriile servicii — ai două copii ale bazei de date în memorie
T5	Builder	Constructorii <code>Teacher</code> și <code>Admin</code> . Uită-te la ordinea parametrilor în cele două, unul lângă altul. Apoi adu-ți aminte de T0.2
T6	Adapter	<code>System.Text.Json</code> pus în spatele lui <code>ITextMapper</code> — schimbi formatul de stocare fără să atingi vreun serviciu
T7	State	<code>Book</code> capătă stări de împrumut: <code>Disponibila</code> → <code>Imprumutata</code> → <code>Restituita/Pierduta</code> , cu tranziții interzise

(T3 Factory Method și T8 Decorator au urcat în **Faza 2** — pe primul îl ceri tu, ca să poată `User` intra în `Repository<T>` .)

Restanțe, oricând între task-uri

- Nu există `.gitignore`. `bin/`, `obj/` și `.vs/` vor ajunge în git — exact ce am curățat la `design-patterns-csharp`.
- Build-ul trece cu **71 de warnings**. Citește-le: cele `CS8604` din `ViewStudent` spun că `Console.ReadLine()` poate întoarce `null` și tu îl folosești direct.
- `Int32.Parse(Console.ReadLine())` în meniuri: orice tastă nenumerică închide aplicația cu `FormatException`.
- `Data/students.txt` nu mai e folosit de nimeni — `users.txt` l-a înlocuit.
- `Console.WriteLine(aux)` rămas din depanare, în `CursTopStudenti` (`ViewStudent.cs:274`).
- `ViewLogin.Logger()` se numește „Logger” dar face login.
- `GetBook` și `FindByName` caută cu `Contains`, nu cu egalitate: „Cartea” găsește „Cartea1”.
- Parolele stau în clar în `users.txt`, iar studenții se loghează doar cu numele, fără parolă.

